

Git, GitHub & Version Control

A Complete Study Notes Guide for Developers

These notes cover what version control is, what Git and GitHub are, why they are used, the difference between them, core concepts, commonly used commands, and a typical workflow — everything needed to build a solid working understanding as a developer.

Contents

1. What is Version Control?
2. Types of Version Control Systems
3. What is Git?
4. Why Git is Used (Purpose & Benefits)
5. What is GitHub?
6. Git vs GitHub — Key Difference
7. Core Git Concepts & Terminology
8. Common Git Commands
9. Typical Git + GitHub Workflow
10. Branching & Merging
11. GitHub Features Beyond Git
12. Best Practices
13. Quick Summary

1. What is Version Control?

Version control (also called source control) is a system that records changes to files over time, so that specific versions can be recalled later. It allows multiple people to work on the same project without overwriting each other's work, and keeps a complete history of every change made to a codebase.

Why it matters

- Tracks every change made to a file — who changed it, when, and why.
- Allows reverting to a previous working version if something breaks.
- Enables multiple developers to work on the same project simultaneously.
- Provides a safe way to experiment (via branches) without affecting the main code.
- Acts as a backup — the entire project history is stored, not just the latest file.

2. Types of Version Control Systems

Type	Description	Examples
Local VCS	Keeps track of file versions in a local database on one machine.	RCS.
Centralized VCS (CVCS)	A single central server holds all versioned files; developers check out files and commit changes back to the server.	SVN, CVS, Perforce.
Distributed VCS (DVCS)	Every developer has a full copy of the repository, including complete history.	Git, Mercurial.

Note: Git is a Distributed VCS — this is a key reason it became the industry standard.

3. What is Git?

Git is a free, open-source, distributed version control system created by Linus Torvalds in 2005 (originally to manage development of the Linux kernel). Git runs locally on a developer's machine and tracks changes to source code over time, allowing developers to save snapshots (commits) of their project and return to them whenever needed.

Key characteristics of Git

- Distributed — every clone of a repository is a full backup with complete history.
- Fast — most operations (commit, diff, log, branch) happen locally, not over a network.
- Branching is lightweight — branches are cheap to create and switch between.
- Data integrity — every commit is checksummed with SHA-1/SHA-256, so history cannot silently change.
- Works offline — you can commit, branch, and view history without an internet connection.

4. Why Git is Used (Purpose & Benefits)

Git solves problems that arise naturally once more than one person works on the same codebase, or once a project needs a reliable history.

Purpose of Git

- **Collaboration:** Multiple developers can work on the same project at the same time without overwriting each other's changes.
- **History tracking:** Every change is recorded with a message, author, and timestamp — useful for auditing and debugging.
- **Backup & recovery:** Since every clone has the full history, losing one copy doesn't lose the project.
- **Experimentation:** Branches let developers try new features or fixes in isolation, without risking the stable code.
- **Merging changes:** Git can intelligently combine work from different people/branches, resolving overlaps where possible.
- **Accountability:** Commits are tied to an author, making it clear who changed what and why.
- **Speed:** Local operations mean Git works quickly even on very large projects and histories.

5. What is GitHub?

GitHub is a cloud-based hosting platform for Git repositories. It is not the same thing as Git — Git is the underlying version control tool, while GitHub is a website/service (owned by Microsoft) that hosts Git repositories online and adds collaboration features on top of Git.

What GitHub adds on top of Git

- A remote, online location to store and share repositories (so teams don't rely on one person's machine).
- Pull Requests (PRs) — a structured way to propose, review, and discuss code changes before merging.
- Issues — for tracking bugs, tasks, and feature requests.
- GitHub Actions — automation for testing, building, and deploying code (CI/CD).
- Access control — manage who can view, edit, or admin a repository.
- Project boards, wikis, discussions, and code review tools for team collaboration.
- A public profile and portfolio — showcasing repositories, contributions, and activity (e.g. a profile like github.com/username).

6. Git vs GitHub — Key Difference

Git	GitHub
A version control tool/software	A hosting platform/service for Git repositories
Installed and runs locally on your computer	Runs on the web/cloud
Works without internet access	Requires internet access
Command-line based (with GUI tools available)	Web interface, plus desktop and mobile apps
Manages history, branches, commits	Adds collaboration: PRs, Issues, Actions, etc.

Open-source project by Linus Torvalds	A company/product, owned by Microsoft
---------------------------------------	---------------------------------------

Note: Analogy: Git is like the engine of a car; GitHub is like the dealership/garage where the car is stored, shown, and shared with others.

7. Core Git Concepts & Terminology

Repository (repo): A folder tracked by Git that stores all files plus their complete change history.

Commit: A saved snapshot of changes, with a message describing what was changed and why.

Branch: An independent line of development; lets you work on a feature without affecting the main code.

Merge: Combining changes from one branch into another.

Clone: Creating a local copy of a remote repository, including all its history.

Fork: A personal copy of someone else's GitHub repository, used to propose changes independently.

Push: Uploading local commits to a remote repository (e.g. GitHub).

Pull: Downloading and merging changes from a remote repository into the local one.

Staging Area (Index): A holding area where changes are placed before they are committed.

HEAD: A pointer to the current commit/branch you are working on.

Remote: A version of the repository hosted elsewhere (e.g. on GitHub), referred to as 'origin' by default.

Pull Request (PR): A GitHub feature to propose changes from one branch/fork and request they be reviewed and merged.

8. Common Git Commands

Command	Purpose
git init	Initialize a new Git repository in a folder
git clone <url>;	Copy an existing remote repository to your local machine
git status	Show changed/staged/untracked files
git add <file>;	Move changes to the staging area
git commit -m "message"	Save staged changes as a new commit
git push	Upload local commits to the remote repository
git pull	Fetch and merge changes from the remote repository
git branch	List, create, or delete branches
git checkout -b <name>;	Create and switch to a new branch
git merge <branch>;	Merge a branch into the current branch
git log	View commit history
git diff	Show differences between commits/working files
git remote -v	View linked remote repositories

9. Typical Git + GitHub Workflow

A common day-to-day workflow when using Git together with GitHub:

- 1. Create/clone a repository — `git clone <url>`
- 2. Create a new branch for your task — `git checkout -b feature-name`
- 3. Make changes to files in your project.
- 4. Stage the changes — `git add .`
- 5. Commit the changes with a message — `git commit -m "Add feature"`
- 6. Push the branch to GitHub — `git push origin feature-name`
- 7. Open a Pull Request on GitHub for review.
- 8. Team reviews, discusses, and approves the PR.
- 9. Merge the PR into the main branch.
- 10. Pull the latest main branch locally — `git pull origin main`

10. Branching & Merging

Branching allows development to diverge from the main line of work (often called **main** or **master**) so that features, fixes, or experiments can be built in isolation. Once the work is ready and tested, it is merged back.

- **main/master:** the primary, usually stable, branch.
- **feature branches:** created for a specific feature or fix, merged back when complete.
- **Merge conflicts:** occur when the same lines of a file are changed differently on two branches; Git flags these for manual resolution.
- **Fast-forward merge:** happens when there are no diverging changes — the branch pointer simply moves forward.

11. GitHub Features Beyond Git

- **Issues:** track bugs, tasks, and feature requests with labels, assignees, and milestones.
- **Pull Requests:** propose and review code changes with inline comments before merging.
- **GitHub Actions:** automate testing, building, and deployment (CI/CD pipelines).
- **GitHub Pages:** host static websites directly from a repository.
- **Projects:** Kanban-style boards to organize and track work.
- **Wiki & README:** document a project directly alongside its code.
- **Profile:** a public developer profile showing repositories and contribution activity, e.g. github.com/username.

12. Best Practices

- Write clear, meaningful commit messages describing *what* and *why*.
- Commit small, logical chunks of work rather than large, mixed changes.
- Use branches for every new feature or fix instead of committing directly to main.
- Pull the latest changes before starting new work to avoid conflicts.
- Use a `.gitignore` file to avoid committing build files, dependencies, or secrets.
- Review code through Pull Requests rather than merging directly.
- Keep repositories organized with a clear README explaining the project.

13. Quick Summary

Version control tracks changes to code over time. **Git** is the distributed version control tool that runs locally and manages this history, branching, and merging. **GitHub** is a cloud platform that hosts Git repositories online and adds collaboration tools such as Pull Requests, Issues, and Actions on top of Git. Together, they let developers track history, collaborate safely, experiment via branches, and ship code with confidence.

Prepared as a developer study reference.